Paddle-MLIR: A Compiler Based on MLIR for Accelerating PaddlePaddle on Edge Intelligence Devices

Longtao Huang
College Electron. Inf. Eng.
State Key Laboratory of Radio Frequency Heterogeneous Integration
Shenzhen University
Shenzhen, China
1849892750@qq.com

Shida Zhong*
College Electron. Inf. Eng.
State Key Laboratory of Radio Frequency Heterogeneous Integration
Shenzhen University
Shenzhen, China
shida.zhong@szu.edu.cn
*Corresponding author

Huihong Liang
College Electron. Inf. Eng.
State Key Laboratory of Radio Frequency Heterogeneous Integration
Shenzhen University
Shenzhen, China
2210434092@email.szu.edu.cn

Tao Yuan
College Electron. Inf. Eng.
Millimeter-Wave Antenna Engineering Research Center
Shenzhen University
Shenzhen, China
yuantao@szu.edu.cn

2024 4th International Conference on Computer Communication and Artificial Intelligence (CCAI) | 979-8-3503-6276-3/24/$31.00 ©2024 IEEE | DOI: 10.1109/CCAI61966.2024.10602841

***Abstract*—AI accelerators provide a promising influence for the deployment of deep learning model on edge devices, and an effective compiler can reduce great workload when deploying the models. Multi-level intermediate representations (MLIR) framework offers a general and efficient solution for compiling and deploying deep learning models on AI hardware because of its reusable and extensible compiler infrastructure. This work utilizes the MLIR framework to compile PaddlePaddle models, and then uses the back-end compiler to generate the target machine codes (bmodel) which are then deployed on an AI hardware--BM1684X. The experimental results show that the compilation of the YOLOv5s model from the PaddlePaddle framework into the MLIR file taking 4.636 seconds without any loss of accuracy, whose time cost is 20% less than compiling the YOLOv5s model from ONNX. The bmodel file, using the FP16 strategy, is executed on the BM1684X processor within 6.508 milliseconds, from loading the model's inputs to outputs. Moreover, employing model in local non-quantization strategy can reduce the entire computing time by 40% with a loss of 1% accuracy.**

Keywords—MLIR, PaddlePaddle, deep learning, AI accelerator

I. INTRODUCTION

Deep Neural Network (DNN) models have been widely used in various tasks such as computer vision, speech recognition, and natural language processing [1-3]. To accelerate the training process of deep learning models, both industry and academia have developed many frameworks such as Caffe, TensorFlow, PyTorch, MXNet, and PaddlePaddle, which have also facilitated the advancement of deep learning in many fields. However, each framework has its unique computational graph representation, which brings a lot of workload to the deployment of deep learning models on target AI hardware.

Meanwhile, since matrix multiplication and high-dimensional tensor convolution operations are complex operations in deep learning models, chip architects are driven to design general-purpose Deep Learning (DL) accelerators that can achieve high performance at low power consumption. Although GPU are still the dominant hardware for training deep models [4], and almost all deep learning frameworks are supported this general-purpose hardware in many tasks, GPU are not the most suitable hardware for deep learning inference. Therefore, many deep learning accelerators, such as Google TPU, Apple Bonic, and SOPHGO TPU are explored and designed, which are more energy-efficient than GPU and can be equipped in edge AI hardware to accelerate the influence of DL models, this makes AI application a better performance.

However, when deploying deep learning models to target AI hardware, a unified intermediate representation is needed for compiling and converting models, thus avoiding the limitation of one framework being able to deploy to only one AI hardware. MLIR, having an extensible compiler architecture, are served as a unified form for compiling models from different frameworks [5]. Reference [6] supports the TensorFlow framework as input and uses MLIR to compile models into formats that can be executed on CPU, GPU, and TPU. Reference [7-8] supports the compilation of machine learning frameworks such as PyTorch and ONNX, although with some limitations on the target hardware. Reference [9] uses the MLIR framework to compile various frameworks such as ONNX, PyTorch, Caffe, etc., supporting a wide range of operators. The transformed MLIR model employs lowering and other operations to generate a bmodel file that can be deployed on TPU devices. Reference [10] uses MLIR to compile PyTorch models and deploys them on AI hardware AB9 [11].

However, there is still a gap in the conversion of models for the PaddlePaddle framework, which needs a set of complex operations and may have potential problems. Based on this, this work proposes an open-source deep learning compiler called Paddle-MLIR, which converts PaddlePaddle models into MLIR textual representations through front-end compilation, and then lowering into machine code through lowering operations. Currently, Paddle-MLIR is validated on CPU and an AI hardware, BM1684X which is developed by SOPHON, as target hardware. This paper will introduce our works by

- presenting the overall design and architecture of Paddle-MLIR.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on September 09,2024 at 12:27:42 UTC from IEEE Xplore. Restrictions apply.

- introducing the process and optimization tricks in fornt-end compilation of the PaddlePaddle model.
- compiling and deploying PaddlePaddle models on BM1684x, which can accelerate the inference of deep learning models.

Our work will be presented as follows. Section II will briefly introduces PaddlePaddle and MLIR framework, as well as the deployed AI hardware, BM1684X. Then the architecture of our proposed compiler Paddle-MLIR will be introduced and the process involving the Paddle conversion will be discussed, as well as some optimization passes in section III. In section IV, the experimental results of compiling and deploying the YOLOv5s model in PaddlePaddle framework will be presented. Finally, section V is the summarization of our work.

II. Background

A. PaddlePaddle

Parallel Distributed Deep Learning (PaddlePaddle) is an open-source deep learning framework developed by Baidu. When constructing models, the neural network structure is composed of a series of Operators and Variables [12-13]. Subsequently, using the "Program" approach, with support for three execution structures: sequential, branching, and looping, it can combine the dynamic description of the calculation process of any complex model. It also supports two programming paradigms in deep learning model development: imperative programming and declarative programming. Network programming paradigm and execution logic are shown in Fig.1. In imperative programming (define and execute), corresponding to the dynamic mode, the network structure is defined and then executed, without the need to compile the network structure into an intermediate representation. Instead, in declarative programming (execution and definition separate), corresponding to the static mode, the network structure described in the "Program" is compiled to intermediate representations first, and then executed. This work focus on the static graph mode of the model when compiling the PaddlePaddle models. In the static graph mode, the representation of the model consists of two parts: one is the pdmodel file describing the network structure, and the other is the pdiparam file saving the weight file.

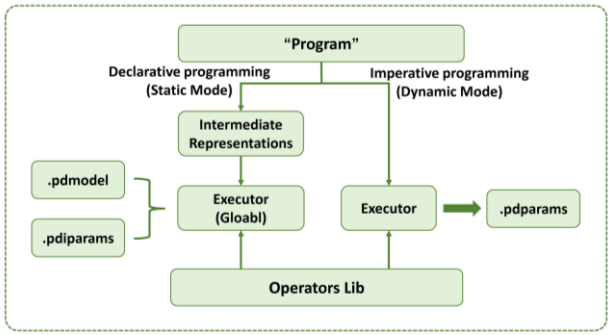


Fig. 1. PaddlePaddle programming paradigm and execution logic.

B. MLIR

MLIR, with its reusability and scalability, presents a new approach to building new domain-specific compilers. MLIR is a subproject of the LLVM project [14], and shares many similarities with the LLVM compilation architecture [15]. It reduces the cost of building domain-specific compilers by facilitating the design and implementation of code generators, translators, and optimizations at different abstraction levels. Its operation can represent many different levels of abstraction and computation processes, from data flow graphs to specific target instructions, and even hardware circuits. However, it is still in the development stage for back-end compilation hardware.

MLIR standardizes its Intermediate Representation (IR) based on Static Single Assignment form, allowing people to represent a series of concepts as first-class operations. In particular, MLIR features operations, attributes, and type interfaces that provide a generic way to interact with IR. Interfaces can be implemented separately from operations and mixed using MLIR's registration mechanism, completely separating IR concepts from transformations. In addition, transformations can be written as combinations of orthogonal local "match and rewrite" primitives. When transforming dialects from higher-level to lower-level, these are often further decomposed into rewrite rules, reducing rules when converting rules to lower-level dialects. Throughout the compilation process, separate dialects can coexist to form a mixed program representation.

C. BM1684X

MLIR is currently in a developmental stage, and its openness and extensibility make it efficient in front-end conversion. However, there is no specialized back-end compilers for MLIR to support target hardware. Therefore, performing our deployment work which is based on compiling PaddlePaddle models using MLIR to generate bmodel models, thus obtaining model files that can run efficiently on specific AI accelerator devices such as the BM1684X processor, integrated with a TPU, which accelerates the computation of deep learning models through dedicated tensor computing units [16].

BM1684X, developed by SOPHON, as a specific TPU-AI accelerator chip, comprises a TPU system and an 8-core A53 subsystem CPU which offer strong computing capabilities. Consequently, BM1684X is able to achieve better results in terms of latency as well as power efficiency when deploying and running deep learning models. Moreover, BM1684X can be applied to various intelligent computing products under SOPHON, implement various deep learning algorithms and support inference in multiple precision types. In this work, MLIR is used for front-end compilation of Paddle models, followed by back-end compilation into machine code to achieve efficient deployment. Considering the aforementioned advantages of BM1684X, this work uses it for verification of our front-end conversion work during deployment.

III. Proposed Methodolgy

This section introduces our compiler, Paddle-MLIR. The overall architecture and some of the transformation strategies in compiling process will be presented here.

A. Architecture of Paddle-MLIR

Fig.2 illustrates the overall architecture of Paddle-MLIR. Firstly, the inputs consist of the pdmodel file describing the

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on September 09,2024 at 12:27:42 UTC from IEEE Xplore. Restrictions apply.

PaddlePaddle model, the pdiparams file containing the weight information, and the input preprocessing parameters required for the conversion(model's input, output and so on). Secondly, the calculation graph information is extracted using the Paddle converter, a Python script, which converts the model into original MLIR text description and saves the weights in the NPZ file. Furthermore, this is followed by optimization and other back-end compilation processes using SOPHON's tpu-mlir, to generate a bmodel file which is the machine code representation of the deep learning model. Finally, the MLIR text description file and the bmodel file are used for inference to validate the effectiveness of our compiler.

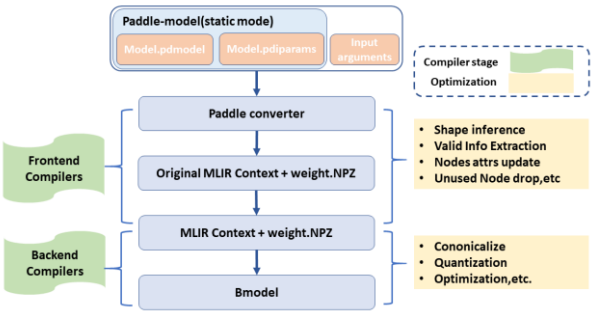


Fig. 2. The compiling architecture of Paddle-MLIR.

In Paddle-MLIR, there are two parts: front-end compilation and back-end compilation. The front-end compilation mainly uses the paddle-converter and the TOP dialect to compile the PaddlePaddle model into a MLIR text representation. The back-end compilation chiefly uses the TPU dialect to convert the MLIR into a bmodel file that can run efficiently on specific hardware accelerators. The TOP and TPU dialects are mainly used to verify the correctness of the paddle dialect compilation.

B. Paddle-Converter

The Paddle-converter is the part of front-end conversion in Paddle-MLIR, which uses the MLIR language to describe the structure of the PaddlePaddle model. Paddle-converter compiles the imported PaddlePaddle model into MLIR to generate table-based operator definitions, which presents all necessary information by using a readable text to describe the operators in the model, while also saving the weight information during the process to an NPZ file. Fig.3 shows the flowchart of the front-end compiling process. First, necessary input arguments are defined via the command line (model input shape, specified model output nodes, model file path, model definition, etc.). Then, the model graph information is extracted by loading the model-related files. Next, the Nodes attrs update strategy is used to update necessary operator information, and the Unused Nodes drop strategy is used to remove some post-processing operators in the model, retaining only the nodes from input to output, simplifying the model. The Shape Inference strategy can get specific shapes of the dynamic shapes of some nodes' inputs and outputs during the process, and then update them in the value dictionary. At this point, all the necessary information has been obtained which describes the model structure from input to output of the entire PaddlePaddle model, thereby converting the paddle model into an initial MLIR text representation and an NPZ file containing the preserved weights. Fig.4 shows the specific contents of the graph information during the conversion process, including tensors containing preserved weights, a dictionary containing preserved node information, and a value dictionary containing the input and output shapes of each node.

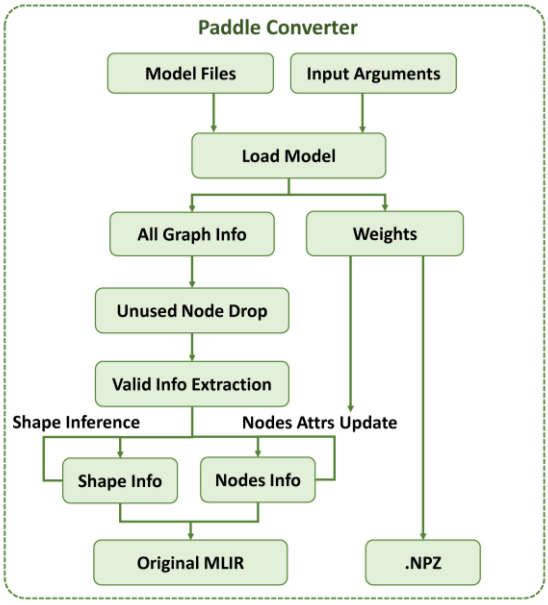


Fig. 3. The front-end compiling process of PaddlePaddle model.

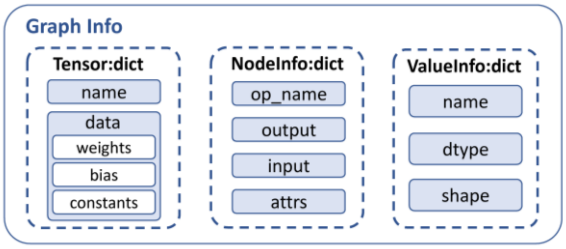


Fig. 4. The graph information of PaddlePaddle's program.

C. Optimization Strategy

Here, This part introduces some strategies used in the compiling process to ensure the correctness of model compilation.

- The “Nodes Attrs Update” strategy involves updating the necessary attributes of operators during the MLIR representation process. In the PaddlePaddle model, the structure is described using a “program”, which encapsulates the description information in blocks. Therefore, when extracting model structure information, it can be obtained from the blocks in the program, which contain values and operators related to the entire computation graph. However, sometimes the attributes of some operators need to be obtained in other ways. For example, the kernel size in the conv2d operator may be missing from the operator information loaded in the model. The process of obtaining relevant attributes for a 2D convolution(conv2d) operator as shown in Fig.5. For the conv2d operator, its shape of kernel should be related to the shape of its input weight. Consequently, the shape of kernel description could be obtained by retrieving the

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on September 09,2024 at 12:27:42 UTC from IEEE Xplore. Restrictions apply.

weight shape of the conv2d operator and update it to its attributes.

- The “Unused Node drop” strategy involves removing unnecessary operator information. For example, if the model includes post-processing steps, this can create obstacles during the conversion process, as there may be complex operators like ‘where’. By removing post-processing operators based on their input and output, the model is simplified.
- “Shape inference” is the process the get the specific shapes of unknown nodes in a paddle model. Fig.6 illustrates the logic of shape inference. The “program” describes the model's structure, and in paddle, each node’s variable is stored under the program's block. Therefore, by traversing the variables in the block, nodes with dynamic shapes are identified, and their names are saved. Random data with the same shape as the input is generated as the model's input which are then passed into the executor to execute the model structure. This process allows for the determination of unknown shapes and updates them in the corresponding node information.

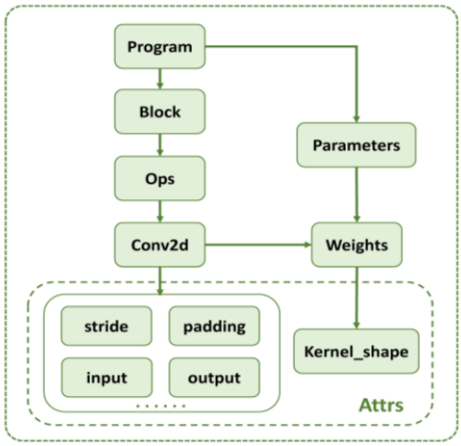


Fig. 5. Conv2d node attrs update.

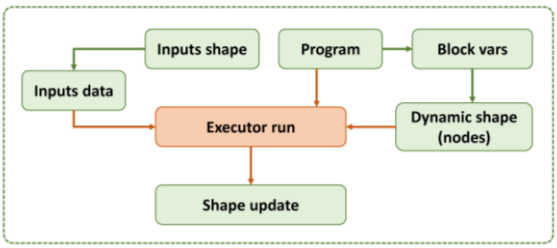


Fig. 6. Dynamic shape inference.

IV. Result Analysis

This work validates Paddle-MLIR using the YOLOv5s model from the PaddlePaddle framework [17]. By configuring the input and output, we performed the model's front-end and back-end conversion on a Docker container and deployed the converted model (bmodel) on the SE7 micro server equipped with the BM1684X processor. The YOLOv5s model consists of 95 operators corresponding to 12 different operator types, with 16.7GFLOPs.

Before presenting the relevant experimental results, let's introduce the strategy of local non-quantization (MIX precision) [18-19]. It's worth noting that for YOLO series models, the last 3 convolutional layers can significantly benefit from using local non-quantization to improve the accuracy of model inference. Therefore, for the last three convolutional layers, this work used the FP32 quantization strategy, while the other layers still used the INT8 quantization strategy.

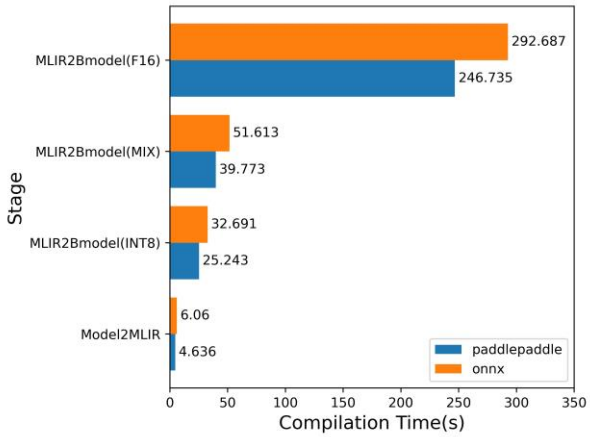


Fig. 7. Compilation time in different stage.

This paper evaluates the compilation time of YOLOv5s using the ONNX and the PaddlePaddle frameworks in Docker (PC host is core (TM) i7-11700K@3.6GHz), which is divided into four stages. The first stage is from the original model to MLIR text representation, the second stage is from MLIR text representation to back-end compilation time with quantization strategy INT8 bmodel, the third stage is from MLIR text representation to back-end compilation time with quantization strategy MIX precision bmodel, and the fourth stage is from MLIR text representation to back-end compilation time with quantization strategy FP16 bmodel. As shown in Fig.7, compiling YOLOv5s model in PaddlePaddle to MLIR cost 4.636 seconds while in ONNX cost 6.06 seconds, which indicates that compiling time PaddlePaddle is 20% less than the conversion time of ONNX [20], which is quite significant.

Mean Average Precision at IoU 50-95 (mAP50-95) is one of the commonly used evaluation metrics in object detection tasks. It calculates the average precision at different Intersection over Union (IoU) thresholds (from 50 to 95 with a step of 5). When calculating mAP50-95, the Average Precision (AP) is computed at different IoU thresholds, and then the average of these AP values is taken to obtain the final mAP50-95 value. This provides a comprehensive assessment of the model's performance, as different IoU thresholds reflect the model's performance at different levels of precision requirements. Table I presents the evaluation results of the model's conversion on the COCO dataset. This work only discusses the accuracy of the corresponding models in different quantization. The mAP50-95 under the FP16 quantization strategy and MLIR conversion is 37.1%, which is consistent with the original model's mAP50-95, indicating that the accuracy remains basically unchanged. However, under the INT8 strategy, the accuracy is 34.8%, a decrease of 2.3%. Therefore, this work used the MIX method to

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on September 09,2024 at 12:27:42 UTC from IEEE Xplore. Restrictions apply.

improve the accuracy, resulting in a mAP50-95 of 36.2%, an increase of 1.4%, which is relatively better result.

TABLE I. COMPARATIVE RESULTS BASED ON COCO DATABASE

Model Files	*Map50-95(%)*
PaddlePaddle	37.1
MLIR	37.1
Bmodel(FP16)	37.1
Bmodel(INT8)	34.8
Bmodel(MIX)	36.1

Furthermore, the back-end-converted bmodel file will be deployed on the SE7 micro server, which is equipped with the BM1684X processor, and the computing time of the model on the SoC under different quantization methods are shown in Fig.8. This work only considers the increase in time ratio, since different computing power will lead to differences in computing time. Using the MIX method increases the time by less than 10% compared to the INT8 quantization method, achieving a favorable trade-off.

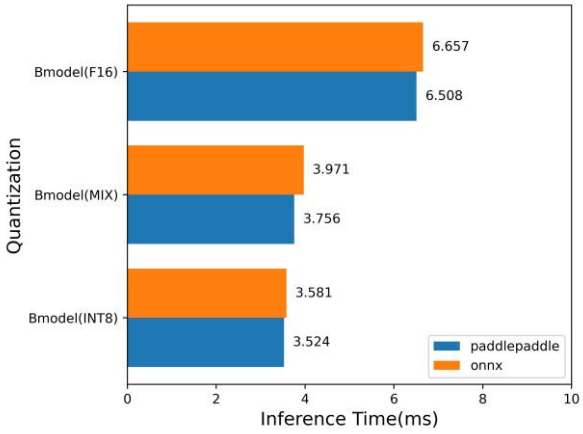


Fig. 8. The inference time of different quantization runs in BM1684X.

V. CONCLUSION

To facilitate the deployment of AI application on edge intelligent devices, this work proposes Paddle-MLIR, which leverages MLIR framework to compile Paddle models and then generates target machine codes. Subsequently, the machine codes are executed on SE7, an edge intelligent device, which is developed by SOPHON. This paper also introduces some optimization strategies in the front-end model conversion to ensure the correctness of compilation. Experimental results show that Paddle-MLIR can compile YOLOv5s from PaddlePaddle into MLIR within 4.636 seconds without any loss in accuracy, and the compilation time is reduced by 20% compared to the ONNX framework. Finally, the compiled files are deployed on SE7, the computing time of YOLOv5s using the FP16 quantization strategy is 6.508 milliseconds. Additionally, by using local non-quantization strategy, the computing time can be further reduced by 40%, with only a 1% loss in accuracy.

REFERENCES

[1] Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. "Imagenet classification with deep convolutional neural networks." Advances in neural information processing systems 25 (2012).

[2] Devlin, Jacob and Chang, Ming-Wei and Lee, Kenton and Toutanova, Kristina. "Bert: Pre-training of deep bidirectional transformers for language understanding." arXiv preprint arXiv:1810.04805 (2018).

[3] Hinton, Geoffrey, et al. "Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups." IEEE Signal processing magazine 29.6 (2012): 82-97.

[4] Oh, Kyoung-Su, and Keechul Jung. "GPU implementation of neural networks." Pattern Recognition 37.6 (2004): 1311-1314.

[5] Lattner, Chris, et al. "MLIR: A compiler infrastructure for the end of Moore's law." arXiv preprint(2020).

[6] Abadi, Martín, et al. "Tensorflow: Large-scale machine learning on heterogeneous distributed systems." arXiv preprint(2016).

[7] LLVM(2024). Torch-MLIR [online]. Available: https://github.com/llvm/torch-mlir

[8] Jin, Tian, et al. "Compiling onnx neural network models using mlir." arXiv preprint arXiv:2008.08272 (2020).

[9] Hu, Pengchao and Lu, Man and Wang, Lei and Jiang, Guoyue. "TPU-MLIR: A Compiler For TPU Using MLIR." arXiv preprint arXiv:2210.15016 (2022).

[10] Kwon, Hyunjeong and Kim, Hyun Mi and Lyuh, Chun-Gi and Kim, Jin-Kyu and Han, Jinho and Kwon, Youngsu. "AIWareK: Compiling PyTorch Model for AI Processor Using MLIR Framework." 2022 IEEE 4th International Conference on Artificial Intelligence Circuits and Systems (AICAS). IEEE, 2022.

[11] Cho, Yong Cheol Peter, et al. "AB9: A neural processor for inference acceleration." ETRI Journal 42.4 (2020): 491-504.

[12] Ma, Yanjun and Yu, Dianhai and Wu, Tian and Wang, Haifeng. "PaddlePaddle: An open-source deep learning platform from industrial practice." Frontiers of Data and Domputing 1.1 (2019): 105-115.

[13] Ao, Yulong, et al. "End-to-end adaptive distributed training on paddlepaddle." arXiv preprint arXiv:2112.02752 (2021).

[14] LLVM: The LLVM Project, 2024. [online]. Available: https://github.com/llvm/llvm-project

[15] Lattner, Chis, and Vikram Adve. "A compilation framework for lifelong program analysis and transformation." CGO. Vol. 4. 2003.

[16] Vasilache, Nicolas, et al. "Composable and modular code generation in MLIR: A structured and retargetable approach to tensor compiler construction." arXiv preprint arXiv:2202.03293 (2022).

[17] PaddlePaddle:PaddleYOLO,2024.[online].Available: https://github.com/PaddlePaddle/PaddleYOLO.

[18] Krishnamoorthi, Raghuraman. "Quantizing deep convolutional networks for efficient inference: A whitepaper." arXiv preprint arXiv:1806.08342 (2018).

[19] Jacob, Benoit, et al. "Quantization and training of neural networks for efficient integer-arithmetic-only inference." Proceedings of the IEEE conference on computer vision and pattern recognition. 2018.

[20] Jin, Tian, et al. "Compiling onnx neural network models using mlir." arXiv preprint arXiv:2008.08272 (2020).

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on September 09,2024 at 12:27:42 UTC from IEEE Xplore. Restrictions apply.